Computer Vision Techniques: Image Restoration, Feature Extraction, Object Detection and Motion Estimation

MCA CCA 3 – Practical Coding Assignment (Computer Vision)

---

1. Cover Page

Project Title: Computer Vision Techniques: Image Restoration, Feature Extraction, Object Detection and Motion Estimation

Submitted in partial fulfilment of the requirements of the course: Computer Vision (MCA)

Submitted by:

- Student Name: [Sunil Ashok Sonawane]
- Roll Number / Registration Number: [1272251168]
- Class / Semester: [SY-MCA/3rd]
- Academic Year: [2025-2027]

Submitted to:

- Faculty Name: [Rahul Inchal]
- Department: [MCA-CS]
- Institution: [MIT-WPU]

Date of Submission: [3/11/2027]

Programming Language: Python 3.13

GitHub Repository: [https://github.com/sunilsonawane29/CV_FaceDetectionProject]

---

4. Acknowledgement

I would like to express my sincere gratitude to my faculty guide, , for teaching the concepts of Computer Vision and for giving me the opportunity to work on this practical assignment. The guidance and feedback provided during the course helped me understand how theoretical ideas such as filtering, feature extraction, object detection and motion estimation are applied in real programs.

I am grateful to the developers of the open-source tools used in this project, namely Python, OpenCV, NumPy, Matplotlib, scikit-image and Ultralytics YOLO. Finally, I thank my family and friends for their support during the preparation of this project.

[Sunil Sonawane]

---

5. Abstract

Computer Vision is the field of computer science that enables computers to extract useful information from images and videos. This project implements and studies four important groups of computer vision techniques using Python, OpenCV, NumPy, Matplotlib, scikit-image and Ultralytics YOLO.

Task 1 (Image Restoration and Segmentation) adds Gaussian noise and salt-and-pepper noise to an image and attempts to remove it using Gaussian and median filters. The restoration quality is measured using PSNR and SSIM. In the experiments, Gaussian noise filtered with a Gaussian filter gave PSNR = 24.10 dB and SSIM = 0.7000, while salt-and-pepper noise filtered with a median filter gave PSNR = 22.96 dB and SSIM = 0.7517. The task also applies binary thresholding, morphological operations (erosion, dilation, opening, closing), Canny edge detection and watershed segmentation.

Task 2 (Feature Extraction and Matching) extracts Histogram of Oriented Gradients (HOG) features (8100 values for each of the two images) and SIFT keypoints (5105 and 13605 keypoints for the two images). Feature matching using BFMatcher produced 240 good matches for the original image pair. A rotation experiment (243 matches) and a resizing experiment (247 matches) showed that the number of matches stayed close to the original value.

Task 3 (Object Detection) compares the classical Haar Cascade face detector (Viola-Jones), which detected 1 face, with the deep learning detector YOLOv8, which detected a person (confidence 0.84) and a tie (confidence 0.66).

Task 4 (Motion Estimation) processes a 20.11-second video of 598 frames (480 × 854 pixels, about 29.74 FPS) using Frame Differencing, Gaussian Mixture Model (MOG2) Background Subtraction and Lucas-Kanade Optical Flow. All 598 frames were processed and three output videos were created. The frame differencing and GMM videos show motion edges and a foreground silhouette respectively. However, the saved GMM comparison panel and the saved Lucas-Kanade still image showed problems during visual inspection, and these are reported honestly as limitations.

Keywords: Computer Vision, Image Restoration, PSNR, SSIM, HOG, SIFT, BFMatcher, Haar Cascade, YOLOv8, Frame Differencing, GMM, MOG2, Lucas-Kanade Optical Flow.

---

6. Table of Contents

(In Microsoft Word, apply Heading 1 and Heading 2 styles and use References → Table of Contents to generate page numbers automatically.)

1. Cover Page

2. Certificate
3. Declaration
4. Acknowledgement
5. Abstract
6. Table of Contents
7. List of Figures
8. List of Tables
9. Introduction
10. Problem Statement
11. Objectives
12. Software and Hardware Requirements
13. Libraries and Tools Used
14. Methodology
15. Task 1: Image Restoration and Segmentation
16. Task 2: Feature Extraction and Matching
17. Task 3: Object Detection
18. Task 4: Motion Estimation
19. Results and Discussion
20. Comparative Analysis
21. Limitations
22. Applications
23. Conclusion
24. Future Scope
25. References
26. Appendices (Submission checklist, screenshot list, file list, demonstration video script)

---

7. List of Figures

Figure	Title	File name
1	Original image	original_image.jpg
2	Image with Gaussian noise	gaussian_noise.jpg
3	Image with salt-and-pepper noise	salt_pepper_noise.jpg
4	Gaussian filtered image	gaussian_filtered.jpg
5	Median filtered image	median_filtered.jpg
6	Task 1 combined results	task1_results.png
7	Binary thresholding	binary_threshold.jpg
8	Canny edge detection	canny_edges.jpg

Figure	Title	File name
9	Erosion	erosion.jpg
10	Dilation	dilation.jpg
11	Opening	opening.jpg
12	Closing	closing.jpg
13	Watershed segmentation	watershed_result.jpg
14	HOG features	hog_features.png
15	SIFT keypoints, image 1	sift_keypoints_image1.jpg
16	SIFT keypoints, image 2	sift_keypoints_image2.jpg
17	Original feature matching	original_feature_matching.jpg
18	Rotated feature matching	rotated_feature_matching.jpg
19	Resized feature matching	resized_feature_matching.jpg
20	Matching comparison	matching_comparison.png
21	Haar Cascade face detection	haar_face_detection.jpg
22	YOLOv8 detection	yolo_detection.jpg
23	Haar vs YOLOv8 comparison	task3_detection_comparison.png
24	Frame differencing result	frame_difference_result.jpg
25	GMM foreground mask	gmm_foreground_mask.jpg
26	Lucas-Kanade optical flow result	optical_flow_result.jpg
27	Task 4 three-method comparison	task4_comparison.png
28	Frame differencing video (frame capture)	frame_differencing_chrome.mp4
29	GMM background subtraction video (frame capture)	gmm_background_subtraction_chrome.mp4
30	Lucas-Kanade optical flow video (frame capture)	lucas_kanade_optical_flow_chrome.mp4

8. List of Tables

Table	Title

Table	Title
1	Software requirements
2	Hardware requirements
3	Libraries and their purpose
4	Task 1: PSNR and SSIM results
5	Task 1: Summary of morphological operations
6	Task 2: HOG and SIFT feature counts
7	Task 2: Feature matching experiments
8	Task 3: Actual detection results
9	Task 3: Haar Cascade vs YOLOv8
10	Task 4: Input video properties
11	Task 4: Verification status of saved outputs
12	Task 4: Comparison of the three techniques
13	Summary of all actual results

---

9. Introduction

Humans understand images with ease. We can tell a person from a chair, notice that a car is moving, and recognise a face in a crowd without thinking. For a computer, an image is only a grid of numbers called pixels. **Computer Vision** is the branch of Artificial Intelligence and Computer Science that develops methods to turn these numbers into useful information.

Computer vision is used in mobile phone cameras, medical imaging, traffic monitoring, security systems, robotics and self-driving cars. A typical computer vision system performs several stages: improving the image quality (pre-processing), separating the important parts (segmentation), describing them (feature extraction), identifying them (detection and recognition) and, for video, following them over time (motion analysis).

This project follows the same pipeline. Four tasks are implemented in Python:

1. **Image Restoration and Segmentation** – removing noise and separating regions of an image.
2. **Feature Extraction and Matching** – describing image content with HOG and SIFT and matching two images.
3. **Object Detection** – finding objects using a classical method (Haar Cascade) and a deep learning method (YOLOv8).
4. **Motion Estimation** – detecting and tracking movement in a video using Frame Differencing, GMM Background Subtraction and Lucas-Kanade Optical Flow.

The project was developed in Visual Studio Code on Windows using Python 3.13 and a virtual environment. This report explains the theory of each technique in simple language, describes how it was used, presents the actual results obtained and honestly states the limitations.

10. Problem Statement

Real-world images and videos are rarely perfect. Images contain noise caused by sensors or transmission, objects must be separated from the background, the same object can look different when it is rotated or resized, objects must be located and named automatically, and in videos moving objects must be detected and tracked.

The problem addressed by this project is:

> *To implement, test and compare standard computer vision techniques for image restoration, segmentation, feature extraction and matching, object detection, and motion estimation, and to evaluate them using numerical measures and visual output.*

11. Objectives

The main objectives of the project are:

1. To add Gaussian and salt-and-pepper noise to an image and restore it using Gaussian and median filters.
2. To evaluate restoration quality using PSNR and SSIM.
3. To apply binary thresholding, morphological operations, Canny edge detection and watershed segmentation.
4. To extract HOG features and SIFT keypoints and descriptors from images.
5. To match features between two images using BFMatcher and study the effect of rotation and resizing.
6. To detect faces with Haar Cascade (Viola-Jones) and objects with YOLOv8, and to compare the two.
7. To implement Frame Differencing, GMM (MOG2) Background Subtraction and Lucas-Kanade Optical Flow on a video.
8. To record the actual results honestly, including limitations and problems.
9. To organise the code, outputs and documentation for submission through GitHub and Google Drive.

12. Software and Hardware Requirements

Table 1: Software requirements

Software	Details
Operating System	Windows (version: [verify, e.g. Windows 11])
Programming Language	Python 3.13
Code Editor	Visual Studio Code

Software	Details
Environment	Python virtual environment (.venv)
Video tool	FFmpeg (used to convert output videos to H.264 MP4)
Media viewing	Windows Media Player and Photos app (to inspect outputs)
Version control	Git and GitHub

Table 2: Hardware requirements

Component	Details
Computer	Laptop (model: [verify])
Processor	[Processor name – verify]
RAM	[RAM size – verify]
Storage	[Free space – verify]
GPU	Not required; the code runs on CPU (state if a GPU was used: [verify])
Camera	Mobile phone camera used to record the input video

Note: Hardware details are placeholders because they were not supplied. Please fill them in from your own system.

13. Libraries and Tools Used

Table 3: Libraries and their purpose

Library	Purpose in this project
OpenCV (cv2)	Reading and writing images and videos, filtering, thresholding, morphology, Canny, watershed, SIFT, BFMatcher, Haar Cascade, MOG2, Lucas-Kanade optical flow
NumPy	Array and matrix calculations, noise generation, image arithmetic
Matplotlib	Creating comparison figures and saving result panels
scikit-image	Computing PSNR and SSIM; HOG feature extraction
Ultralytics YOLO	Loading and running the pretrained YOLOv8 model

Version numbers of each library should be copied from your `requirements.txt` file (run `pip freeze` in the virtual environment). They are not invented here.

14. Methodology

The project followed a step-by-step approach. Each task was developed in its own Python script and its outputs were saved in a separate folder, so that results could be inspected independently.

Project structure (as used in this project):

- `src/` – Python source files (for example `task1_restoration.py`, `task3_object_detection.py`, `task4_motion_estimation.py`, `convert_task4_videos.py`)
- `outputs/unit3/` – Task 1 outputs (restoration and segmentation images)
- `outputs/unit4/` – Task 2 outputs (HOG, SIFT and matching images)
- `outputs/unit5/` – Task 3 detection outputs
- `outputs/unit5/task4/` – Task 4 comparison images, masks, videos and results text file

General workflow for every task:

1. Load the input image or video using OpenCV.
2. Convert to the required format (for example grayscale or float).
3. Apply the algorithm.
4. Save result images or videos.
5. Calculate numerical measures where available.
6. Inspect the outputs visually and record observations.

Mathematical measures used.

Mean Squared Error (MSE) measures the average squared difference between the original image I and the processed image K of size m × n:

```
MSE = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\left[I(i,j)-K(i,j)\right]^2
```

Peak Signal-to-Noise Ratio (PSNR) is calculated from MSE. For 8-bit images, MAX = 255:

```
PSNR = 10\log_{10}\left(\frac{MAX^2}{MSE}\right)\ \text{dB}
```

A higher PSNR means the processed image is numerically closer to the original.

Structural Similarity Index (SSIM) compares luminance, contrast and structure between two images:

```
SSIM(x,y)=\frac{(2\mu_x\mu_y+C_1)(2\sigma_{xy}+C_2)}{(\mu_x^2+\mu_y^2+C_1)(\sigma_x^2+\sigma_y^2+C_2)}
```

SSIM lies between -1 and 1, and a value close to 1 means the images are structurally very similar.

Intersection over Union (IoU) is used in object detection to measure how much a predicted box overlaps the true box:

```
IoU=\frac{\text{Area of Overlap}}{\text{Area of Union}}
```

IoU is explained here for theory only. No ground-truth boxes were prepared for this project, so IoU was **not calculated**.

Honesty note on methodology. Exact parameter values (for example noise strength, kernel sizes, thresholds, number of tracked points) are defined inside the source files. Where this report mentions them, they are marked **[verify from code]** so that you can copy the correct value from your own script.